МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Государственный Университет Управления

Кафедра: Институт Информационных Систем

Лабораторная работа № 9

Способы представления графов и алгоритмы обхода в ширину и в глубину

Дисциплина: Теория алгоритмов

Направление подготовки: 38.03.05 «Бизнес-информатика»

Язык реализации: Python 3.10+

Вариант: 14

Выполнил: студент группы БИ 2-1 Лапшенков Григорий Витальевич

Проверил: [должность, ФИО преподавателя]

[Город], 2026

1. Цель работы и задание варианта

1.1. Цель работы

Изучить способы представления графов в памяти, реализовать алгоритмы обхода в ширину и в глубину и экспериментально установить, как выбор представления влияет на расход памяти и время решения типовых задач на графах.

1.2. Задание варианта 14

Таблица 1. Исходные данные варианта

Параметр	Значение
Тип графа	P — разреженный, $m \approx 3n$, рёбра случайны
Ориентированность	ориентированный граф (дуги)
Размеры n	500; 2000; 5000
Задача варианта	обнаружение цикла
Дополнительное задание	И7 — анализ схемы бизнес-процесса: недостижимые шаги и циклы, указывающие на ошибки проектирования

Требуется: реализовать матрицу смежности и списки смежности с единым интерфейсом (`add_edge`, `neighbors`, `has_edge`), итеративные BFS и DFS; проверить их на малом графе; измерить время обхода и расход памяти на графах варианта; решить задачу обнаружения цикла; построить графики; выполнить задание И7.

2. Представления графа и теоретические оценки

Граф $G = (V, E)$, $|V| = n$, $|E| = m$; в ориентированном графе дуга (u, v) ведёт из u в v.

Таблица 2. Теоретические характеристики представлений

Характеристика	Матрица смежности	Списки смежности
Память	$\Theta(n^2)$	$\Theta(n + m)$
Проверка наличия дуги (u, v)	$\Theta(1)$	$\Theta(\deg^+ u)$
Перебор соседей вершины	$\Theta(n)$	$\Theta(\deg^+ v)$
Полный обход (BFS, DFS)	$\Theta(n^2)$	$\Theta(n + m)$
Обнаружение цикла (DFS)	$\Theta(n^2)$	$\Theta(n + m)$

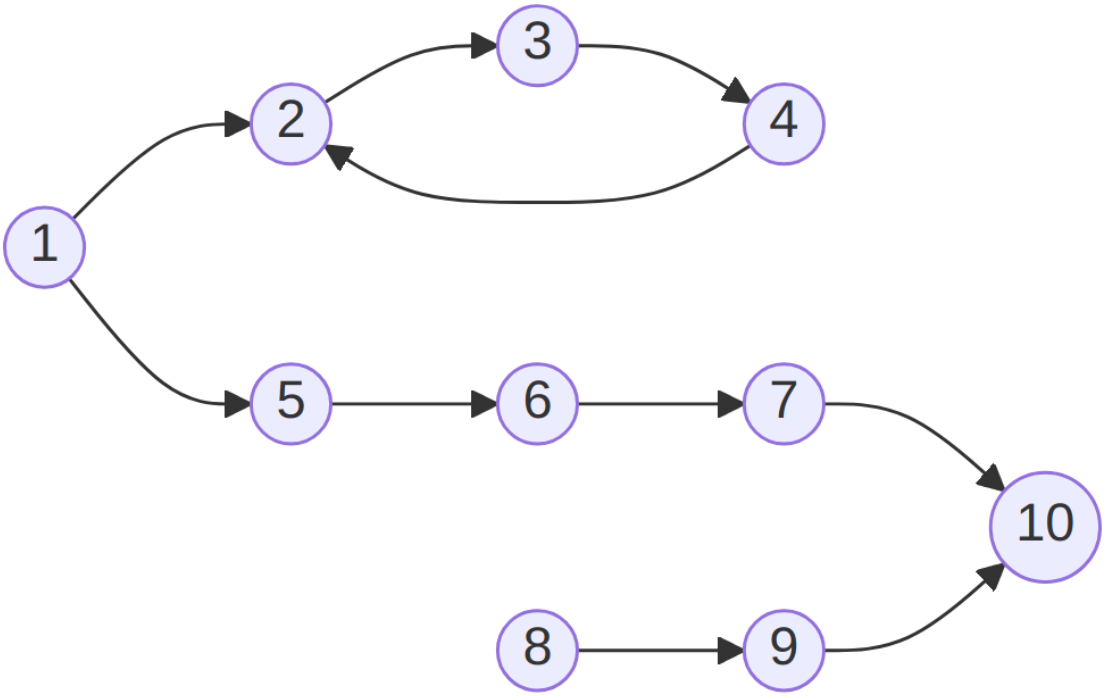
Для графа варианта $m = 3n$, поэтому списки требуют $\Theta(n)$ памяти и времени, а матрица — $\Theta(n^2)$: при $n = 5000$ это $25 \cdot 10^6$ ячеек против $2 \cdot 10^4$ элементов списков.

Обнаружение цикла в ориентированном графе. При обходе в глубину каждая вершина имеет один из трёх цветов: 0 (не посещена), 1 («в обработке», то есть находится в стеке текущего пути), 2 (обработана). Если при просмотре дуги $v \to u$ вершина u имеет цвет 1, то u лежит на пути от начала текущего обхода до v, и дуга $v \to u$ замыкает цикл $u \to \ldots \to v \to u$. Обратно, любой цикл при обходе в глубину обязательно порождает такую дугу (для первой посещённой вершины цикла). Если все вершины обработаны без обнаружения дуги в «серую» вершину, цикла нет. Трудоёмкость $\Theta(n + m)$ для списков и $\Theta(n^2)$ для матрицы. Сам цикл восстанавливается по массиву предков `parent`. Использована итеративная реализация с явным стеком итераторов: рекурсивный DFS на цепочке из 10^5 вершин вызвал бы переполнение стека вызовов.

Независимая проверка. Для контроля корректности используется алгоритм Кана (удаление вершин с нулевой входящей степенью): если после него остались вершины, цикл есть. Результаты двух алгоритмов сверяются на случайных графах.

3. Тестовый граф и результат его обхода

Ориентированный граф из 10 вершин с одним циклом и отдельной цепочкой, недостижимой из вершины 1:



Результаты программы (вершины нумеруются с 1):

- BFS от вершины 1: порядок 1, 2, 5, 3, 6, 4, 7, 10; расстояния до вершин 1…10: 0, 1, 2, 3, 1, 2, 3, -1, -1, 4 (значение −1 — вершина недостижима: 8 и 9);
- DFS от вершины 1: порядок 1, 2, 3, 4, 5, 6, 7, 10;
- найденный цикл: 2 → 3 → 4 → 2.

BFS посещает вершины по уровням (расстояния 0, 1, 2, 3, 4), DFS идёт вглубь по ветви 2 → 3 → 4, затем возвращается и обходит ветвь 5 → 6 → 7 → 10. Вершины 8 и 9 в обход не попали, так как в них нет дуг из достижимых вершин. Дуга $4 \to 2$ ведёт в вершину, находящуюся «в обработке», — она и замыкает цикл 2 → 3 → 4 → 2.

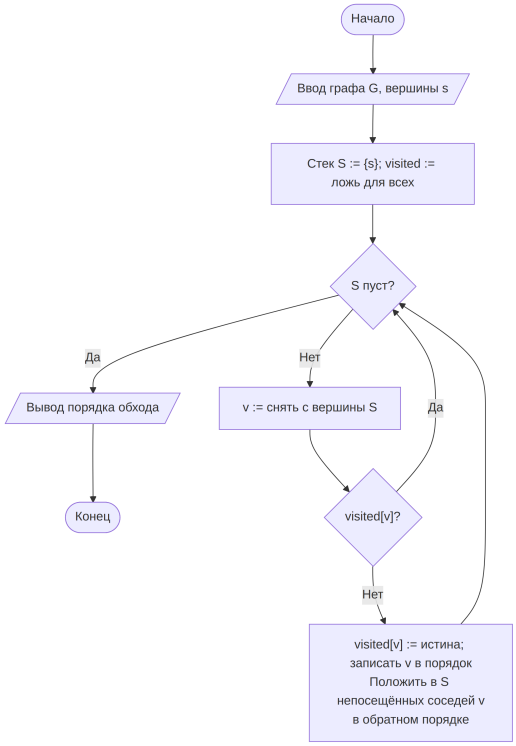
Самопроверка. На 300 случайных графах (2…25 вершин; часть графов строилась заведомо ациклической; цикл содержали 131 графов) результаты `find_cycle` совпали с алгоритмом Кана для обоих представлений, а каждый найденный цикл проверен по существованию всех его дуг.

4. Блок-схемы алгоритмов

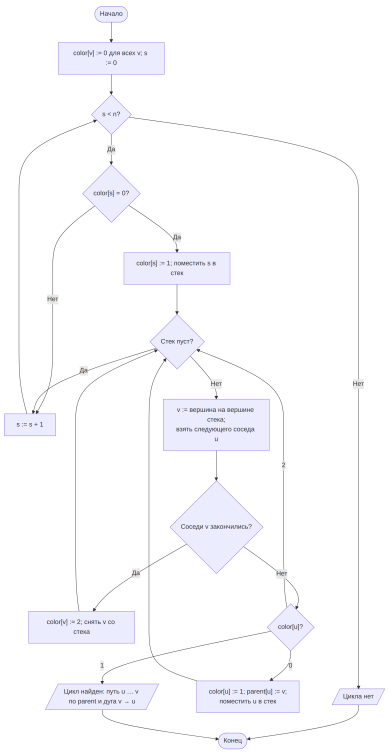
4.1. Обход в ширину (BFS)



4.2. Итеративный обход в глубину (DFS)



4.3. Обнаружение цикла (DFS с тремя цветами)



5. Листинг программы

Программа содержит оба представления, BFS, итеративный DFS, поиск цикла и проверку его корректности, алгоритм Кана для сверки, генератор случайных разреженных и ациклических графов, замеры времени и памяти, анализ схемы бизнес-процесса и построение графиков.

```
"""
Лабораторная работа № 9. Вариант 14.
Матрица смежности и списки смежности для ориентированных графов; BFS, DFS;
обнаружение цикла. Разреженные графы (m ≈ 3n), n = 500, 2000, 5000.
Дополнительное задание И7: анализ схемы бизнес-процесса (недостижимые шаги, циклы).
"""
import gc
import json
import random
import statistics
import sys
import time
from collections import deque


# ---------------------------------------------------- представления графа
class MatrixGraph:
    """Ориентированный граф на матрице смежности. Память Θ(n²)."""

    def __init__(self, n):
        self.n = n
        self.matrix = [[0] * n for _ in range(n)]

    def add_edge(self, u, v):
        self.matrix[u][v] = 1                  # дуга u → v

    def has_edge(self, u, v):
```

```
        return self.matrix[u][v] == 1

    def neighbors(self, v):
        row = self.matrix[v]
        return [u for u in range(self.n) if row[u]]


class ListGraph:
    """Ориентированный граф на списках смежности. Память θ(n + m)."""

    def __init__(self, n):
        self.n = n
        self.adj = [[] for _ in range(n)]

    def add_edge(self, u, v):
        self.adj[u].append(v)

    def has_edge(self, u, v):
        return v in self.adj[u]

    def neighbors(self, v):
        return self.adj[v]


# ---------------------------------------------------------------- обходы
def bfs(graph, start):
    """Обход в ширину. Возвращает (порядок обхода, расстояния, предки)."""
    n = graph.n
    dist, parent, order = [-1] * n, [-1] * n, []
    dist[start] = 0
    queue = deque([start])
    while queue:
        v = queue.popleft()
        order.append(v)
        for u in graph.neighbors(v):
            if dist[u] == -1:
                dist[u] = dist[v] + 1
                parent[u] = v
                queue.append(u)
    return order, dist, parent


def dfs(graph, start):
    """Итеративный обход в глубину. Возвращает порядок посещения."""
    visited = [False] * graph.n
    order, stack = [], [start]
    while stack:
        v = stack.pop()
        if visited[v]:
            continue
        visited[v] = True
        order.append(v)
        for u in reversed(graph.neighbors(v)):
            if not visited[u]:
                stack.append(u)
    return order


# ----------------------------------------- задача варианта: поиск цикла
def find_cycle(graph):
```

```
    """Обнаружение цикла в ориентированном графе обходом в глубину с тремя цветами:
    0 — не посещена, 1 — «в обработке» (в стеке), 2 — обработана.
    Дуга в вершину цвета 1 замыкает цикл. Возвращает список вершин цикла или None.
    Итеративная реализация (явный стек итераторов): нет ограничения глубины рекурсии."""
    n = graph.n
    color, parent = [0] * n, [-1] * n
    for s in range(n):
        if color[s]:
            continue
        color[s] = 1
        stack = [(s, iter(graph.neighbors(s)))]
        while stack:
            v, it = stack[-1]
            for u in it:
                if color[u] == 0:
                    color[u] = 1
                    parent[u] = v
                    stack.append((u, iter(graph.neighbors(u))))
                    break
                if color[u] == 1:                    # обратная дуга v → u
                    cycle, x = [v], v
                    while x != u:
                        x = parent[x]
                        cycle.append(x)
                    cycle.reverse()                  # u → … → v, дуга v → u замыкает цикл
                    return cycle
            else:
                color[v] = 2
                stack.pop()
    return None


def check_cycle(graph, cycle):
    """Проверка найденного цикла: все дуги, включая замыкающую, существуют."""
    return all(graph.has_edge(cycle[i], cycle[(i + 1) % len(cycle)]) for i in range(len(cycle)))


def has_cycle_kahn(graph):
    """Независимая проверка (алгоритм Кана): остались вершины — значит, есть цикл."""
    n = graph.n
    indeg = [0] * n
    for v in range(n):
        for u in graph.neighbors(v):
            indeg[u] += 1
    queue = deque(v for v in range(n) if indeg[v] == 0)
    done = 0
    while queue:
        v = queue.popleft()
        done += 1
        for u in graph.neighbors(v):
            indeg[u] -= 1
            if indeg[u] == 0:
                queue.append(u)
    return done < n


# ---------------------------------------------------------- генерация
def random_arcs(n, m, rng, acyclic=False):
    """m различных дуг (u, v), u != v. При acyclic=True все дуги направлены
    от вершины с меньшим «рангом» к большему — граф ациклический."""
```

```
    rank = list(range(n))
    rng.shuffle(rank)
    m = min(m, n * (n - 1) // 2 if acyclic else n * (n - 1))
    arcs = set()
    while len(arcs) < m:
        u, v = rng.randrange(n), rng.randrange(n)
        if u == v:
            continue
        if acyclic and rank[u] > rank[v]:
            u, v = v, u
        arcs.add((u, v))
    return sorted(arcs)


def build(cls, n, arcs):
    g = cls(n)
    for u, v in arcs:
        g.add_edge(u, v)
    return g


def deep_sizeof(obj):
    """Расход памяти списка списков, байт; каждое вложенное число учитывается один раз
    (малые числа в CPython разделяются между ячейками)."""
    total = sys.getsizeof(obj)
    unique = {}
    for row in obj:
        total += sys.getsizeof(row)
        for x in row:
            if id(x) not in unique:
                unique[id(x)] = sys.getsizeof(x)
    return total + sum(unique.values())


def timed(func, *args, repeats=3):
    times = []
    for _ in range(repeats):
        gc.collect()
        t0 = time.perf_counter()
        result = func(*args)
        times.append(time.perf_counter() - t0)
    return statistics.median(times), result


# ---------------------------------------------------- проверка корректности
def self_test(trials=300):
    """Сверка find_cycle с алгоритмом Кана на случайных малых графах обоих представлений."""
    rng = random.Random(9)
    with_cycle = 0
    for _ in range(trials):
        n = rng.randint(2, 25)
        m = rng.randint(0, min(3 * n, n * (n - 1)))
        arcs = random_arcs(n, m, rng, acyclic=rng.random() < 0.4)
        for cls in (MatrixGraph, ListGraph):
            g = build(cls, n, arcs)
            cyc = find_cycle(g)
            assert (cyc is not None) == has_cycle_kahn(g), (n, arcs)
            if cyc is not None:
                assert check_cycle(g, cyc), cyc
        with_cycle += cyc is not None
```

```
    return trials, with_cycle


# ---------------------------------------- И7: схема бизнес-процесса
def strongly_connected(graph):
    """Компоненты сильной связности (алгоритм Косарайю, итеративный)."""
    n = graph.n
    order, seen = [], [False] * n
    for s in range(n):
        if seen[s]:
            continue
        seen[s] = True
        stack = [(s, iter(graph.neighbors(s)))]
        while stack:
            v, it = stack[-1]
            for u in it:
                if not seen[u]:
                    seen[u] = True
                    stack.append((u, iter(graph.neighbors(u))))
                    break
            else:
                order.append(v)
                stack.pop()
    rev = ListGraph(n)
    for v in range(n):
        for u in graph.neighbors(v):
            rev.add_edge(u, v)
    comp, comps = [-1] * n, []
    for s in reversed(order):
        if comp[s] != -1:
            continue
        members, stack = [], [s]
        comp[s] = len(comps)
        while stack:
            v = stack.pop()
            members.append(v)
            for u in rev.neighbors(v):
                if comp[u] == -1:
                    comp[u] = len(comps)
                    stack.append(u)
        comps.append(sorted(members))
    return comps, rev


STEPS = ["Начало", "Приём заказа", "Проверка данных клиента", "Проверка наличия товара",
         "Запрос уточнения у клиента", "Резервирование товара", "Выставление счёта",
         "Ожидание оплаты", "Проверка оплаты", "Напоминание об оплате", "Комплектация",
         "Доставка", "Закрытие заказа", "Конец", "Отмена заказа", "Возврат средств",
         "Пересчёт скидки", "Эскалация менеджеру", "Повторная проверка менеджером",
         "Уведомление клиента", "Ожидание ответа клиента"]
ARCS = [(0, 1), (1, 2), (2, 3), (2, 4), (4, 20), (20, 4), (3, 5), (3, 14), (5, 6), (6, 7),
        (7, 8), (8, 10), (8, 9), (9, 7), (10, 11), (11, 12), (12, 13), (14, 13), (11, 19),
        (15, 12), (16, 6), (17, 18), (18, 17), (18, 13)]
START, FINAL = 0, 13


def analyze_process():
    g = build(ListGraph, len(STEPS), ARCS)
    _, dist, _ = bfs(g, START)
    reachable = [v for v in range(g.n) if dist[v] != -1]
```

```
    unreachable = [v for v in range(g.n) if dist[v] == -1]
    dead_ends = [v for v in range(g.n) if not g.neighbors(v) and v != FINAL]
    comps, rev = strongly_connected(g)
    cyclic = []
    for c in comps:
        if len(c) > 1:
            inside = set(c)
            has_exit = any(u not in inside for v in c for u in g.neighbors(v))
            cyclic.append({"steps": c, "exit": has_exit, "reachable": c[0] in reachable})
    _, back, _ = bfs(rev, FINAL)                    # из каких шагов достижим «Конец»
    cannot_finish = [v for v in reachable if back[v] == -1]
    return {"reachable": reachable, "unreachable": unreachable, "dead_ends": dead_ends,
            "cyclic": cyclic, "cannot_finish": cannot_finish}


# ---------------------------------------------------------------- графики
def make_plots(res):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    rows = res["rows"]
    ns = [r["n"] for r in rows]

    fig, axes = plt.subplots(1, 2, figsize=(11, 4.2))
    for ax, key, title in ((axes[0], "bfs", "Обход в ширину (BFS) из вершины 0"),
                           (axes[1], "cyc_dag", "Обнаружение цикла в ациклическом графе (полный
↪   обход)")):
        ax.loglog(ns, [r[key]["matrix"] for r in rows], "o-", label="матрица смежности")
        ax.loglog(ns, [r[key]["list"] for r in rows], "s-", label="списки смежности")
        ax.set_xlabel("число вершин n"); ax.set_ylabel("время, с"); ax.set_title(title,
↪   fontsize=10)
        ax.grid(True, which="both", alpha=.3); ax.legend()
    plt.tight_layout(); plt.savefig("img/lr9_time.png", dpi=170); plt.close()

    plt.figure(figsize=(6.6, 4.2))
    x = range(len(ns))
    plt.bar([i - 0.2 for i in x], [r["mem"]["matrix"] / 2 ** 20 for r in rows], 0.4, label="матрица
↪   смежности")
    plt.bar([i + 0.2 for i in x], [r["mem"]["list"] / 2 ** 20 for r in rows], 0.4, label="списки
↪   смежности")
    plt.yscale("log"); plt.xticks(list(x), ns); plt.xlabel("число вершин n"); plt.ylabel("память,
↪   МБ")
    plt.title("Расход памяти представлениями графа"); plt.grid(True, axis="y", which="both",
↪   alpha=.3); plt.legend()
    plt.tight_layout(); plt.savefig("img/lr9_memory.png", dpi=170); plt.close()

    pr = res["prob"]
    plt.figure(figsize=(6.6, 4.2))
    plt.plot([p["c"] for p in pr], [p["share"] * 100 for p in pr], "o-")
    plt.xlabel("средняя исходящая степень m / n"); plt.ylabel("доля графов с циклом, %")
    plt.title("Вероятность цикла в случайном орграфе (n = 500)")
    plt.grid(True, alpha=.3); plt.tight_layout(); plt.savefig("img/lr9_prob.png", dpi=170);
↪   plt.close()


# ---------------------------------------------------------------- main
if __name__ == "__main__":
    res = {}
    res["selftest"] = self_test()
    print("Самопроверка: графов", res["selftest"][0], ", из них с циклом", res["selftest"][1])
```

```
# --- 1. Малый граф: обходы и цикл
small_arcs = [(0, 1), (1, 2), (2, 3), (3, 1), (0, 4), (4, 5), (5, 6), (7, 8), (8, 9), (6, 9)]
small = build(ListGraph, 10, small_arcs)
o_bfs, dist, _ = bfs(small, 0)
res["small"] = {"arcs": small_arcs, "bfs": [v + 1 for v in o_bfs], "dist": dist,
                "dfs": [v + 1 for v in dfs(small, 0)],
                "cycle": [v + 1 for v in find_cycle(small)]}
print(res["small"])

# --- 2. Сравнение представлений, m ≈ 3n
rng = random.Random(14)
res["rows"] = []
print(f"{'n':>6}{'m':>8}{'BFS матр.':>11}{'BFS спис.':>11}{'цикл DAG матр.':>16}{'цикл DAG
 ↪  спис.':>16}{'МБ матр.':>10}{'МБ спис.':>10}")
for n in (500, 2000, 5000):
    m = 3 * n
    arcs = random_arcs(n, m, rng)
    dag_arcs = random_arcs(n, m, rng, acyclic=True)
    gm, gl = build(MatrixGraph, n, arcs), build(ListGraph, n, arcs)
    dm, dl = build(MatrixGraph, n, dag_arcs), build(ListGraph, n, dag_arcs)
    row = {"n": n, "m": m, "bfs": {}, "cyc_rand": {}, "cyc_dag": {}, "mem": {}}
    for key, g_m, g_l in (("bfs", gm, gl), ("cyc_rand", gm, gl), ("cyc_dag", dm, dl)):
        func = (lambda g: bfs(g, 0)) if key == "bfs" else find_cycle
        reps = 3
        tm, rm = timed(func, g_m, repeats=reps)
        tl, rl = timed(func, g_l, repeats=reps)
        row[key] = {"matrix": tm, "list": tl}
        if key == "bfs":
            assert rm[1] == rl[1]
            row["reach"] = sum(1 for d in rm[1] if d != -1)
        else:
            assert (rm is None) == (rl is None)
            if key == "cyc_rand":
                assert rl is not None and check_cycle(gl, rl)
                row["cycle_len"] = len(rl)
            else:
                assert rl is None and not has_cycle_kahn(dl)
    row["mem"] = {"matrix": deep_sizeof(gm.matrix), "list": deep_sizeof(gl.adj)}
    res["rows"].append(row)
    print(f"{n:>6}{m:>8}{row['bfs']['matrix']:>11.4f}{row['bfs']['list']:>11.4f}"
          f"{row['cyc_dag']['matrix']:>16.4f}{row['cyc_dag']['list']:>16.4f}"
          f"{row['mem']['matrix'] / 2 ** 20:>10.1f}{row['mem']['list'] / 2 ** 20:>10.2f}",
           ↪  flush=True)
    del gm, dm

# --- 3. Вероятность цикла в зависимости от m / n (n = 500)
res["prob"] = []
n, trials = 500, 200
for c in (0.3, 0.6, 0.9, 1.0, 1.2, 1.5, 2.0, 3.0):
    cnt = 0
    for _ in range(trials):
        g = build(ListGraph, n, random_arcs(n, int(c * n), rng))
        cyc = find_cycle(g)
        assert (cyc is not None) == has_cycle_kahn(g)
        cnt += cyc is not None
    res["prob"].append({"c": c, "share": cnt / trials})
print(res["prob"])

# --- 4. И7: схема бизнес-процесса
```

```
    res["process"] = {"steps": STEPS, "arcs": ARCS, "analysis": analyze_process()}
    print(res["process"]["analysis"])

    with open("res/lr9.json", "w", encoding="utf-8") as f:
        json.dump(res, f, ensure_ascii=False)
    make_plots(res)
```

6. Таблицы измерений и графики

6.1. Условия измерений

Параметр	Значение
Операционная система	Linux 6.18 (x86_64), виртуальная машина
Процессор	Intel Xeon, 2,10 ГГц, 1 виртуальное ядро
Оперативная память	4 ГБ
Интерпретатор	CPython 3.12.3
Способ замера времени	`time.perf_counter()`, перед каждым прогоном вызывается `gc.collect()`

Графы: случайные ориентированные без петель и кратных дуг, $m = 3n$; для замеров обнаружения цикла в графе без цикла использован случайный ациклический граф с тем же m (все дуги направлены от меньшего «ранга» вершины к большему). Для каждого замера выполнено 3 прогона, приведена медиана. Память — функция `deep_sizeof`: размер внешнего списка, всех вложенных списков и всех различных вложенных объектов, каждый объект учтён один раз (малые числа 0 и 1 в CPython разделяются между ячейками). Измерения выполнены на одном и том же наборе дуг для обоих представлений.

6.2. Время обхода в ширину

Таблица 3. Время BFS из вершины 1 (полный обход достижимой части)

n	m	Достижимо из вершины 1	BFS, матрица, с	BFS, списки, с	Отношение
500	1 500	479	0,00367	0,00012	29,9
2 000	6 000	1 886	0,06749	0,00072	93,3
5 000	15 000	4 705	0,39605	0,00198	200,0

Таблица 4. Рост времени BFS при увеличении n

Переход	Рост n	Матрица (опыт)	Теория $\Theta(n^2)$	Списки (опыт)	Теория $\Theta(n+m)$
500 → 2000	4	18,4	16	5,9	4
2000 → 5000	2,5	5,9	6,25	2,7	2,5

Для матрицы время растёт почти по закону n^2 (в 18,4 раза при росте n в 4 раза и в 5,9 раза при росте в 2,5 раза). Рост для списков получился несколько большим теоретического (5,9 и 2,7 против 4 и 2,5): времена BFS по спискам при $n = 500$ составляют десятки микросекунд, поэтому на них заметно влияют постоянные накладные расходы (вероятная причина; отдельно не проверялась). При $n = 5000$ списки быстрее матрицы в 200 раз.

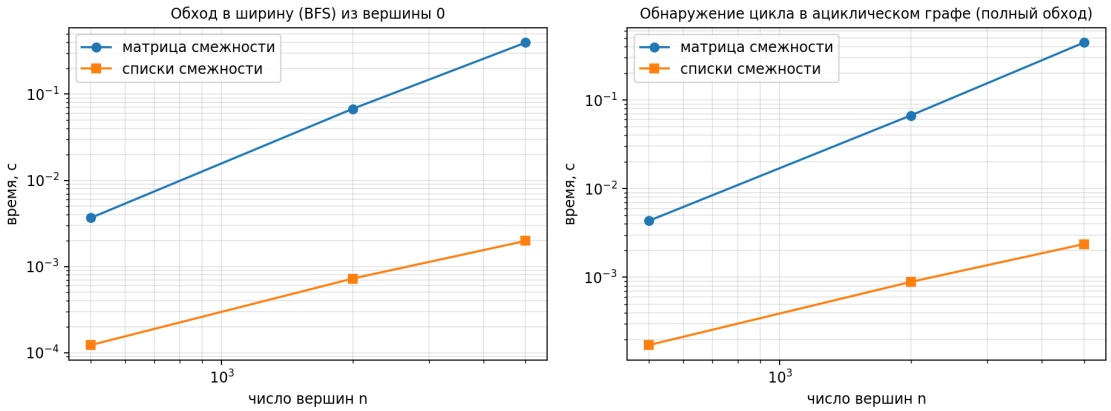


Рис. 1. Время обхода и обнаружения цикла для двух представлений (логарифмические шкалы)

6.3. Расход памяти

Таблица 5. Расход памяти представлениями графа

n	Матрица смежности, МБ	Списки смежности, МБ	Отношение
500	1,94	0,07	26
2 000	30,64	0,34	90
5 000	191,04	0,87	220

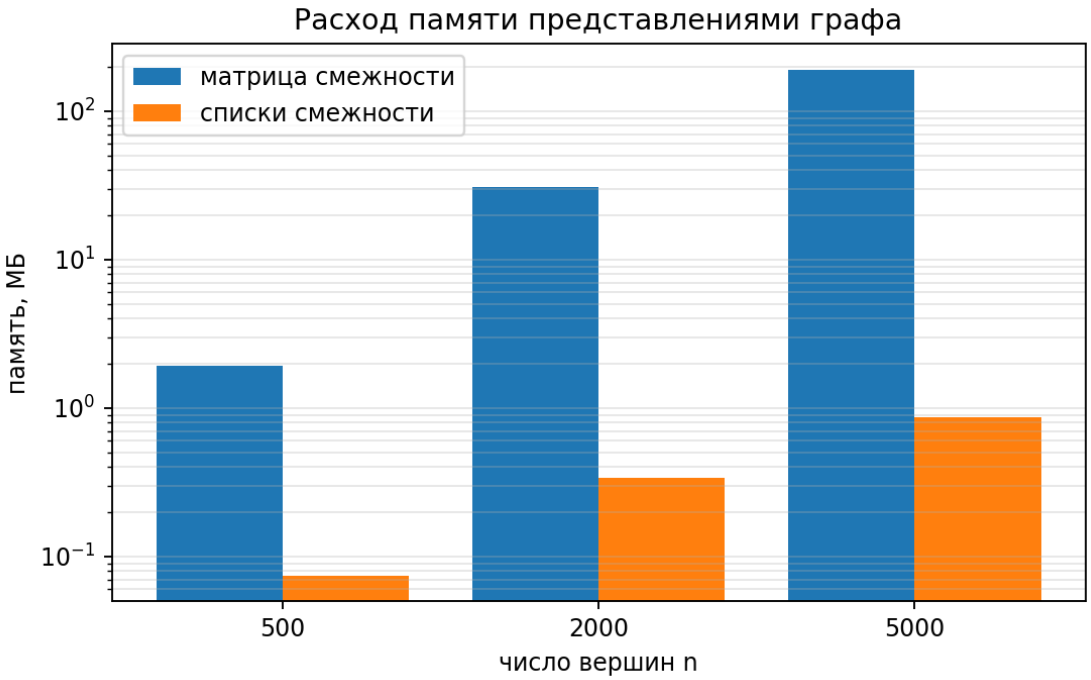


Рис. 2. Расход памяти (логарифмическая шкала)

Память матрицы растёт как n^2, списков — линейно: при $n = 5000$ матрица занимает 191 МБ, списки — 0,87 МБ (в 220 раз меньше).

7. Результат решения задачи варианта: обнаружение цикла

Таблица 6. Обнаружение цикла (ориентированные разреженные графы)

n	Граф	Результат	Матрица, с	Списки, с
500	случайный, $m = 3n$	цикл найден (длина 28)	0,00027	0,00003
	ациклический, $m = 3n$	цикла нет (полный обход)	0,00434	0,00017
2 000	случайный, $m = 3n$	цикл найден (длина 2)	0,00151	0,00005
	ациклический, $m = 3n$	цикла нет (полный обход)	0,06691	0,00089
5 000	случайный, $m = 3n$	цикл найден (длина 15)	0,00173	0,00007
	ациклический, $m = 3n$	цикла нет (полный обход)	0,44325	0,00237

Случайный граф с $m = 3n$ при всех размерах содержит цикл; поиск останавливается на первом найденном цикле (длины 28, 2 и 15 соответственно), поэтому время мало и для матрицы (от 0,3 до 1,7 мс), а различие между представлениями определяется лишь стоимостью перебора соседей посещённых вершин. Найденные циклы проверены функцией `check_cycle` (все дуги существуют).

Ациклический граф — наихудший случай для задачи: поиск должен обойти всё, прежде чем вывести «цикла нет». Время такого полного обхода сопоставимо со временем BFS: для матрицы растёт как n^2 (0,0043 с, 0,067 с и 0,44 с), для списков — линейно (0,00017; 0,00089; 0,0024 с); при $n = 5000$ списки быстрее в 187 раз. Ответ «цикла нет» подтверждён алгоритмом Кана.

Вероятность наличия цикла. Чтобы понять, насколько типичен цикл, для $n = 500$ по 200 случайным графам определена доля графов с циклом при разной средней исходящей степени m/n (ответы `find_cycle` на каждом графе сверены с алгоритмом Кана):

Таблица 7. Доля случайных ориентированных графов с циклом ($n = 500$**, 200 графов на строку)**

m/n	Доля графов с циклом, %
0,3	3,0
0,6	29,0
0,9	66,5
1,0	84,0
1,2	98,0
1,5	100,0
2,0	100,0
3,0	100,0



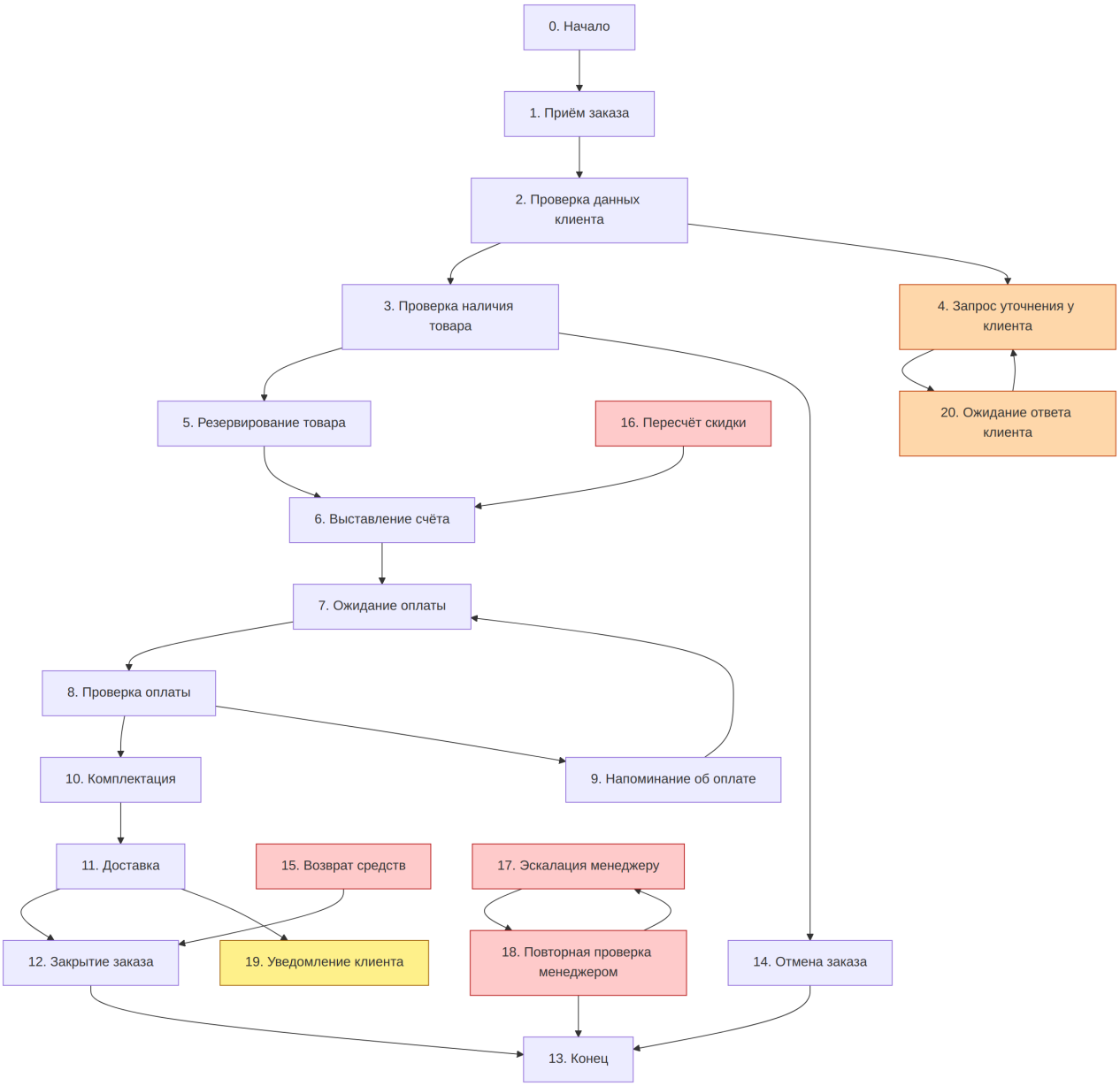
Рис. 3. Доля графов с циклом в зависимости от средней исходящей степени

При $m/n = 0{,}3$ цикл есть лишь у 3,0 % графов, при $m/n = 1$ — у 84,0 %, а начиная с $m/n \approx 1{,}5$ — у всех испытанных графов. Для графа варианта ($m/n = 3$) наличие цикла практически неизбежно, поэтому на таких данных вопрос «есть ли цикл» почти всегда решается положительно, а содержательна задача поиска самого цикла или проверки схемы, спроектированной как ациклическая.

8. Дополнительное задание И7: анализ схемы бизнес-процесса

8.1. Постановка

Схема процесса обработки заказа в интернет-магазине представлена ориентированным графом: вершины — шаги процесса, дуги — допустимые переходы. Шаг 0 («Начало») — вход, шаг 13 («Конец») — единственный допустимый выход. Требуется найти ошибки проектирования: недостижимые шаги, циклы (в частности, циклы без выхода), тупики и шаги, из которых процесс не может завершиться. Недостижимость определяется обходом BFS из шага 0, циклы — разложением на компоненты сильной связности (алгоритм Косарайю: два обхода DFS, второй по транспонированному графу); компонента из двух и более шагов содержит цикл. Достижимость конца определяется BFS по транспонированному графу от шага «Конец». Схема содержит 21 шаг и 24 перехода (составлена для демонстрации; закрашены дефектные шаги: красные — недостижимые, оранжевые — цикл без выхода, жёлтые — тупик).



8.2. Результаты анализа

Таблица 8. Найденные дефекты схемы

Дефект	Шаги	Пояснение
Недостижимые шаги	15, 16, 17, 18 (Возврат средств; Пересчёт скидки; Эскалация менеджеру; Повторная проверка менеджером)	в них нельзя попасть из шага «Начало»
Цикл без выхода («ловушка»)	4, 20 (Запрос уточнения у клиента, Ожидание ответа клиента)	из цикла нет дуг наружу: процесс не может завершиться
Цикл с выходом	7, 8, 9 (Ожидание оплаты, Проверка оплаты, Напоминание об оплате)	повторные напоминания допустимы, но число повторов не ограничено
Цикл среди недостижимых шагов	17, 18 (Эскалация менеджеру, Повторная проверка менеджером)	не влияет на работу процесса, пока шаги недостижимы
Тупик (нет исходящих дуг)	19 (Уведомление клиента)	шаг не является конечным, но из него некуда идти
Шаги, из которых «Конец» недостижим	4, 19, 20	объединение ловушки и тупика

Достижимы 17 шагов из 21. Циклов обнаружено три: по результатам разложения на компоненты сильной связности нетривиальные компоненты — {4, 20}, {7, 8, 9} и {17, 18}.

8.3. Рекомендации по исправлению

- Цикл {4, 20} — «ловушка»: клиент, не ответивший на запрос уточнения, остаётся в процессе навсегда. Нужно добавить выход (отмена заказа по тайм-ауту) в шаг «Ожидание ответа клиента».
- Цикл {7, 8, 9} (напоминания об оплате) имеет выход через успешную оплату, но не ограничен по числу повторов; следует добавить счётчик попыток и переход к отмене заказа.
- Шаги 15 («Возврат средств»), 16 («Пересчёт скидки»), 17 и 18 недостижимы: либо добавить к ним входящие переходы (например, возврат средств после отмены оплаченного заказа), либо удалить из схемы; цикл {17, 18} требует выхода к шагу «Конец».
- Шаг 19 («Уведомление клиента») — тупик: добавить переход к шагу «Конец» (или к закрытию заказа).

После исправления ловушки и тупика все достижимые шаги будут иметь путь к шагу «Конец».

9. Выводы

1. На разреженных ориентированных графах списки смежности превосходят матрицу и по времени, и по памяти: при $n = 5000$, $m = 3n$ BFS по спискам быстрее в 200 раз, память меньше в 220 раз (0,87 МБ против 191 МБ). Время обхода матрицы растёт как $\Theta(n^2)$, списков — как $\Theta(n + m)$.
2. Обнаружение цикла обходом в глубину с тремя цветами корректно (результат совпал с алгоритмом Кана на 300 малых и на 1600 случайных графах), имеет трудоёмкость $\Theta(n + m)$ для списков и $\Theta(n^2)$ для матрицы; при $n = 5000$ в ациклическом графе (полный обход) списки быстрее в 187 раз. Итеративная реализация не зависит от глубины рекурсии.
3. Случайный граф со средней исходящей степенью 3 практически всегда содержит цикл; доля графов с циклом приближается к 100 % уже при $m/n \approx 1{,}5$. Поэтому вопрос о цикле в реальных схемах решает не вероятность, а проектное намерение: схема, которая должна быть ациклической, требует проверки.
4. Задача И7 показывает практическое применение: один проход BFS и разложение на компоненты сильной связности находят недостижимые шаги, тупики и «ловушки» в схеме процесса за $\Theta(n + m)$. В примере выявлены 4 недостижимых шага, один тупик и 3 цикла, один из которых — цикл без выхода.
5. **Рекомендация по выбору представления.** Для графов вида «схема процесса», «сеть связей» (разреженных) выбирать списки смежности. Матрица оправдана для

плотных графов и при частых проверках наличия конкретной дуги за $\Theta(1)$; в Python к тому же каждая ячейка матрицы — отдельная ссылка (8 байт), поэтому расход памяти велик даже при малом числе дуг.

10. Ответы на контрольные вопросы

1. Дайте определение графа. Чем ориентированный граф отличается от неориентированного? Граф $G = (V, E)$ — пара множеств: вершин V и рёбер $E \subseteq V \times V$. В неориентированном графе ребро $\{u, v\}$ соединяет вершины без выделенного направления, а в ориентированном дуга (u, v) идёт из u в v, и наличие (u, v) не означает наличия (v, u). В этой работе граф ориентированный: матрица смежности несимметрична ($A[u][v] = 1$ только для дуги $u \to v$), в списках дуга хранится один раз.

2. Какой граф называется разреженным, какой — плотным? Приведите примеры из предметной области. Граф разрежен, если $m = O(n)$, и плотен, если $m = \Theta(n^2)$. Разрежены схемы бизнес-процессов (в разобранном примере 21 шаг и 24 перехода), оргструктуры, сети поставок; плотными бывают таблицы «все связаны со всеми» (например, матрица расстояний между офисами). Граф варианта — разреженный: $m = 3n$.

3. Сравните матрицу смежности и списки смежности по памяти и по времени перебора соседей. Матрица: память $\Theta(n^2)$, перебор соседей $\Theta(n)$; списки: память $\Theta(n + m)$, перебор $\Theta(\deg v)$. Опыт ($n = 5000$, $m = 15\,000$): 191 МБ против 0,87 МБ, BFS по матрице в 200 раз медленнее.

4. Когда матрица смежности предпочтительнее списков? Для плотных графов ($m = \Theta(n^2)$, затраты памяти на матрицу сравнимы с затратами на списки), при частых проверках «есть ли дуга (u, v)» (за $\Theta(1)$ против $\Theta(\deg u)$) и в алгоритмах, использующих матричные операции (Флойд — Уоршелл, возведение матрицы смежности в степень для подсчёта путей).

5. Опишите алгоритм BFS. Какая структура данных используется и почему? Стартовая вершина помечается и помещается в очередь; пока очередь не пуста, извлекается вершина, и все её непомеченные соседи помечаются (с расстоянием +1 и предком) и ставятся в очередь. Используется очередь (FIFO): она обеспечивает обработку вершин в порядке возрастания расстояния от старта (сначала весь уровень k, затем уровень $k + 1$). Трудоёмкость $\Theta(n + m)$ для списков.

6. Почему BFS находит кратчайшие пути в невзвешенном графе, а DFS — нет? Очередь гарантирует, что вершина впервые достигается по пути минимальной длины: все вершины уровня k извлекаются раньше вершин уровня $k + 1$, поэтому первое достижение вершины происходит по кратчайшему пути. DFS идёт вглубь и достигает вершину по первому встреченному пути, который может быть длиннее. Пример: дуги $1 \to 2$, $2 \to 3$, $1 \to 3$; DFS, пойдя сначала в вершину 2, достигнет вершины 3 по пути $1 \to 2 \to 3$ длиной 2, тогда как BFS найдёт путь $1 \to 3$ длиной 1.

7. Как восстановить сам путь, а не только его длину? В BFS при помечивании вершины u запоминается её предок `parent[u] = v`. Путь до вершины t получают, двигаясь по `parent` от t к старту и разворачивая список. В тестовом графе путь от вершины 1 до вершины 10: 10 → 7 → 6 → 5 → 1, то есть 1 → 5 → 6 → 7 → 10 (длина 4). Аналогично по `parent` восстанавливается цикл в `find_cycle`.

8. Как с помощью DFS обнаружить цикл в ориентированном графе? Раскрасить вершины в три цвета (не посещена, в обработке, обработана). Дуга из текущей вершины в вершину цвета «в обработке» замыкает цикл (раздел 2). Дуга в уже обработанную вершину цикла не создаёт. В неориентированном графе достаточно встретить посещённую вершину, не являющуюся непосредственным предком. Трудоёмкость $\Theta(n + m)$ для списков.

9. Почему рекурсивный DFS опасен на больших графах? Как этого избежать? Глубина рекурсии равна длине самого глубокого пути: для цепочки из 10^5 вершин стек вызовов Python (по умолчанию около 1000) переполнится — `RecursionError`. Избежать можно итеративной реализацией с явным стеком (так сделано в работе: стек итераторов в `find_cycle`, стек вершин в `dfs`), либо увеличением лимита рекурсии с риском аварийного завершения интерпретатора.

10. Как с помощью обхода найти все компоненты связности? Какова трудоёмкость? Запускать BFS (или DFS) из каждой ещё не посещённой вершины: все

вершины одного запуска образуют компоненту. Суммарная трудоёмкость $\Theta(n + m)$ для списков (каждая вершина и дуга обрабатываются один раз). Для ориентированного графа таким способом находятся компоненты слабой связности (дуги считаются неориентированными); компоненты сильной связности находит алгоритм Косарайю (два обхода DFS), использованный в задании И7: {4, 20}, {7, 8, 9} и {17, 18}.